C / C++ / Python Programming Mastery

A standards-aligned curriculum for fundamentals, DSA, competitive programming and interviews

The Talent Grid Candidate Revision Guide

This curriculum is designed to move candidates from basic programming to interview-grade problem solving using C, C++ and Python in a proper sequence.

Outcome: candidates learn syntax, logic building, DSA, algorithmic thinking, debugging, mini projects and mock interview explanation.

Learn programming in the correct sequence - from syntax to interview problem solving.

Curriculum Roadmap

The sequence below follows how programming should be learned in a standards-aligned way: first syntax and control flow, then memory and modularity, then data structures, algorithms, projects and interviews.

Learning Journey

Area	C Standard Practice	C++ Standard Practice	Python Standard Practice
Program entry	int main() and return 0	int main() and fast I/O	if __name__ == '__main__'
Input	scanf/fgets with validation	cin/cout, getline	input(), sys.stdin for large input
Memory	Manual stack/heap awareness	RAII, references, smart pointers	Object references and garbage collection
Collections	Arrays and custom structs	STL containers	list, dict, set, tuple, collections

Area	C Standard Practice	C++ Standard Practice	Python Standard Practice
Error handling	Return codes and checks	Exceptions plus checks	try/except and validation
Coding style	Clear functions and headers	STL-aware, const-correct where useful	PEP 8 inspired, readable names
Interview focus	Pointers, arrays, memory, strings	STL, OOP, DSA speed	Clean logic, libraries, data handling

Complete Module Sequence

The table gives the full curriculum in the right learning order. Each module is mapped to what must be practiced in C, C++ and Python.

Module	Concepts	Language Mapping	Candidate Outcome
1. Programming Environment	Compiler, interpreter, source code, executable, IDE, terminal, command line, CPU, memory, input and output.	C: gcc, main(), printf, scanf | C++: g++, iostream, cin, cout | Python: interpreter, print, input	Candidate can compile, run and explain how a program executes.
2. Variables and Data Types	Declaration, initialization, integer, float, double, char, string, bool, constants, type casting and naming conventions.	C: int/float/double/char/const | C++: string/bool/const | Python: int/float/str/bool/dynamic typing	Candidate can choose the correct data type and avoid basic overflow/type errors.
3. Input and Output	Read user input, format output, handle spaces, lines, numbers and strings.	C: scanf/printf | C++: cin/cout/getline | Python: input/print/f-string	Candidate can write clean console programs and handle input constraints.

Module	Concepts	Language Mapping	Candidate Outcome
4. Operators and Expressions	Arithmetic, relational, logical, assignment, increment, decrement, bitwise operators and precedence.	C/C++: &&, ||, !, ++, --, &, |, ^, <<, >> | Python: and/or/not, **, //	Candidate can convert formulas and conditions into correct expressions.

Module	Concepts	Language Mapping	Candidate Outcome
5. Decision Making	if, else, nested if, else-if ladder, switch-case and ternary decisions.	C/C++: if, else, switch, ?: | Python: if, elif, else	Candidate can model business rules such as tax slab, grade, discount or eligibility logic.
6. Loops	for, while, do-while, nested loops, break, continue, infinite loops and dry run.	C/C++: for/while/do-while | Python: for range, while	Candidate can repeat logic safely and solve counting, summation and pattern problems.

Module	Concepts	Language Mapping	Candidate Outcome
7. Functions	Function declaration, definition, calling, parameters, return value, scope, local/global variables and recursion basics.	C/C++: return type and prototype | Python: def and return	Candidate can decompose code into reusable blocks and improve readability.
8. Arrays, Lists and Matrices	Indexing, traversal, searching, insertion, deletion, 1D arrays, 2D arrays and matrices.	C: fixed arrays | C++: arrays and vector | Python: list	Candidate can solve collection-based problems and understand memory layout basics.
9. Strings	Character arrays, string objects, traversal, length, comparison, substring, split, join, palindrome and anagram.	C: char[], strlen, strcmp | C++: string | Python: str and slicing	Candidate can handle text input, parsing and interview string problems.

Module	Concepts	Language Mapping	Candidate Outcome
10. Pointers, References and Memory	Address, pointer, dereference, stack, heap, dynamic memory, call by value and call by reference.	C: *, &, malloc/free | C++: references, new/delete, smart pointers | Python: object references	Candidate understands memory-sensitive bugs, mutation and parameter passing.
11. Structures, Classes and OOP	Struct, class, object, constructor, encapsulation, inheritance, polymorphism and abstraction.	C: struct | C++: class, constructor, inheritance, virtual | Python: class, __init__, inheritance	Candidate can model real entities such as students, products, employees and accounts.
12. File Handling	Open, read, write, append, close, CSV files, logs and file errors.	C: FILE*, fopen/fclose | C++: ifstream/ofstream | Python: with open(), csv	Candidate can persist data beyond program execution.

Module	Concepts	Language Mapping	Candidate Outcome
13. Error Handling and Debugging	Syntax error, runtime error, logical error, boundary case, defensive programming and exception handling.	C: manual checks | C++: try/catch | Python: try/except/finally	Candidate can find defects, handle invalid input and write robust programs.
14. Standard Libraries	Reusable library functions and containers for faster, safer implementation.	C: stdio, stdlib, string, math | C++: STL vector/map/set/stack/queue/algorithm | Python: collections, itertools, heapq, bisect	Candidate can avoid reinventing common operations and write interview-ready code faster.

Module	Concepts	Language Mapping	Candidate Outcome
15. Data Structures	Array, string, stack, queue, linked list, hash table, tree, heap, graph and trie.	C: manual implementation | C++: STL plus manual | Python: built-ins plus manual	Candidate can choose the right structure for time and space efficiency.

Module	Concepts	Language Mapping	Candidate Outcome
16. Algorithms	Searching, sorting, recursion, backtracking, two pointers, sliding window, hashing, greedy, DP and graph algorithms.	Same concepts across C, C++ and Python with language-specific implementation style.	Candidate can solve coding interview and competitive programming problems.
17. Complexity Analysis	Big O, best case, average case, worst case, time complexity and space complexity.	Language independent; used to compare brute force and optimized solutions.	Candidate can explain why a solution passes constraints.
18. Projects and Mock Interviews	Mini projects, timed tests, code reviews, resume explanation and final mock interviews.	C: records and memory projects | C++: OOP/STL projects | Python: automation/data projects	Candidate becomes ready to demonstrate skill, not just syntax knowledge.

Phase 1 - Programming Foundation

This phase builds comfort with the machine, language syntax and predictable program execution.

- Understand compiler versus interpreter and how a program moves from source code to execution.
- Write first programs in C, C++ and Python using proper entry points and input/output.
- Use variables and data types correctly for numbers, text and logical values.
- Practice business-style mini problems such as marksheet, bill calculation, simple interest and temperature conversion.

Validation: candidate can write 20 small programs without help, dry-run each line and explain expected output.

Foundation Flow

Phase 2 - Logic Building

Logic building is the bridge between knowing syntax and solving real coding problems.

- Use if-else and switch-case for rule-based decisions.
- Use loops for repetition and nested loops for pattern and matrix problems.
- Break a problem into input, processing, conditions, iteration and output.
- Handle boundary cases such as zero, negative values, empty input and maximum constraints.

Practice Area	Examples	What can go wrong	How to validate
Conditions	Grade, tax slab, ATM withdrawal	Wrong branch order or missed edge case	Dry-run all ranges
Loops	Factorial, prime, Fibonacci	Infinite loop or off-by-one	Check first, middle and last values
Nested loops	Patterns and matrices	Wrong row/column boundary	Trace row and column indexes

Phase 3 - Functions, Recursion and Modularity

Functions make code reusable, testable and interview-ready. Recursion teaches candidates how the call stack works.

Concept	Meaning	Used for	Risk if weak
Function	A named block that performs one task	Reusable logic such as add, prime check, GCD	Large unreadable code
Parameter	Value sent to a function	Customizing the function input	Wrong result due to wrong argument
Return value	Output sent back by a function	Using function result elsewhere	Printing instead of returning
Scope	Where a variable is visible	Avoiding accidental changes	Name conflicts and hidden bugs
Recursion	Function calling itself	Tree traversal, backtracking, divide and conquer	Missing base case or stack overflow

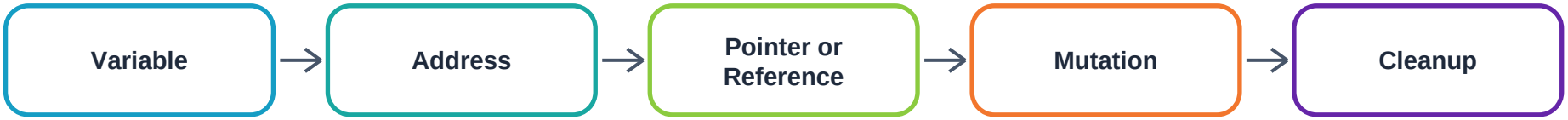
Recommended practice: prime check, factorial, reverse number, GCD, LCM, palindrome and recursive sum of digits.

Phase 4 - Collections, Strings and Memory

Arrays, strings and memory concepts are essential for C/C++ interviews and still important conceptually in Python.

Topic	C	C++	Python
Array/List	Fixed-size contiguous memory	array and vector	list object
String	char array and string.h	std::string	str with slicing and methods
Matrix	2D array	vector>	list of lists
Pointer	Direct address handling	Pointer plus reference	Object reference concept
Dynamic memory	malloc/calloc/free	new/delete and smart pointer	Garbage collector handles memory

Memory Thinking



Phase 5 - Object-Oriented Programming

OOP means Object-Oriented Programming. It helps candidates model real-world entities like students, accounts, products and employees.

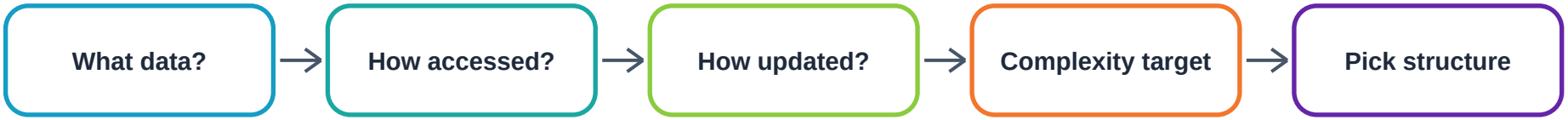
OOP Concept	Simple Meaning	Real Example	Validation
Class	Blueprint	BankAccount class	Can define attributes and methods
Object	Actual instance	Savings account of one customer	Can create multiple objects
Encapsulation	Keep data and behavior together	Deposit/withdraw methods protect balance	Invalid state is prevented
Inheritance	Reuse parent behavior	Employee -> Manager	Common fields not duplicated
Polymorphism	Same action, different behavior	Shape area for circle/rectangle	Correct method selected
Abstraction	Hide internal complexity	ATM hides banking backend	User sees simple interface

Phase 6 - Data Structures

Data structures decide how data is stored, accessed and updated. The right data structure reduces time complexity and improves code clarity.

Sequence	Data Structure	When to use	Key operations
1	Array/List	Ordered indexed data	Access, update, traverse
2	Stack	Last-in-first-out problems	Push, pop, top
3	Queue	First-in-first-out processing	Enqueue, dequeue
4	Linked List	Frequent insert/delete with nodes	Insert, delete, reverse
5	Hash Table	Fast lookup by key	Insert, find, count
6	Tree	Hierarchical data	Traversal, search
7	Heap	Priority-based retrieval	Push, pop min/max
8	Graph	Network relationships	BFS, DFS, shortest path
9	Trie	Prefix search	Insert word, search prefix

Choosing a Data Structure



Phase 7 - Algorithms and Problem Patterns

Algorithms are reusable problem-solving patterns. Candidates should learn them as patterns, not isolated questions.

Pattern	What it means	Typical problems	Validation metric
Linear Search	Check every element one by one.	Find item, count frequency, first match.	O(n), correct for unsorted data.
Binary Search	Repeatedly cut sorted search space in half.	Lower bound, rotated array, answer search.	O(log n), no infinite loop.
Sorting	Arrange data for faster lookup or greedy decisions.	Merge intervals, rank records, count inversions.	Correct order and complexity.
Two Pointers	Use two indexes to reduce nested loops.	Pair sum, remove duplicates, container area.	O(n), pointer movement justified.

Pattern	What it means	Typical problems	Validation metric
Sliding Window	Maintain a moving range over data.	Max sum size K, longest substring, min window.	Window invariant remains valid.
Hashing	Use key-value lookup for fast access.	Two sum, anagrams, subarray sum K.	Average O(1) lookup, handles collisions conceptually.
Recursion	Function solves smaller version of the same problem.	Factorial, tree traversal, divide and conquer.	Base case exists, stack depth safe.

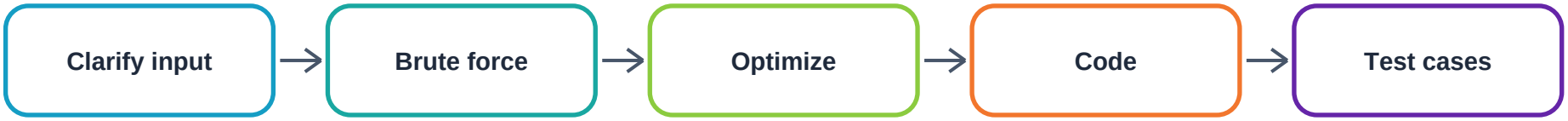
Pattern	What it means	Typical problems	Validation metric
Backtracking	Try choices, undo, and explore alternatives.	Subsets, permutations, N-Queens, Sudoku.	All valid states explored once.

Pattern	What it means	Typical problems	Validation metric
Greedy	Make best local choice when it leads to global optimum.	Activity selection, meeting rooms, gas station.	Proof or counterexample reasoning.
Dynamic Programming	Store answers to repeated subproblems.	Knapsack, LCS, LIS, edit distance.	State, transition and base case clear.
Graph Algorithms	Model relationships as nodes and edges.	BFS, DFS, course schedule, shortest path.	Visited logic and edge cases correct.

Competitive Programming and Interview Standards

Candidates must write fast, clean and constraint-aware code. A good answer includes approach, complexity, edge cases and final code.

Coding Interview Answer Flow



Skill	Candidate should say	Candidate should do
Approach	I will first explain brute force, then optimize.	Write steps before code.
Complexity	The solution is O(n) time and O(n) space.	Connect complexity to constraints.
Edge cases	I will test empty input, one element and duplicates.	Run sample and custom tests.
Code quality	I will keep logic modular and readable.	Use functions and meaningful names.

Standard Code Templates

C competitive programming starter

```
#include <stdio.h>

int main() {
    int t;
    scanf("%d", &t);

    while (t--) {
        int n;
        scanf("%d", &n);
        printf("%d\n", n);
    }

    return 0;
}
```

C++ competitive programming starter

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    cin >> t;
    while (t--) {
        int n;
        cin >> n;
        cout << n << '\n';
    }
    return 0;
}
```

Python competitive programming starter

```
import sys

def solve():
    data = sys.stdin.read().split()
    index = 0
    t = int(data[index])
    index += 1

    answers = []
    for _ in range(t):
        n = int(data[index])
        index += 1
        answers.append(str(n))

    print("\n".join(answers))

if __name__ == "__main__":
    solve()
```

90-Day Training Plan

This schedule can be used as a The Talent Grid programming track for college students, freshers and candidates preparing for service-based and product-based coding rounds.

Week	Focus	C/C++/Python Output
1	Programming setup, compiler/interpreter, variables, data types, I/O	10 console programs in each selected language
2	Operators, conditions, loops and pattern printing	Number logic and nested loop problem set
3	Functions, scope, recursion basics	Reusable function library for common math/string tasks
4	Arrays, lists, strings and basic sorting/searching	Array/string interview starter problems

Week	Focus	C/C++/Python Output
5	Pointers, references, memory model, structures/classes	Memory exercises and OOP mini examples
6	Stack, queue, linked list and hashing	Manual and library-based implementations
7	Binary search, two pointers, sliding window, recursion	Pattern-wise coding drill
8	Trees, heaps, graphs, BFS and DFS	Traversal and graph representation problems

Week	Focus	C/C++/Python Output
9	Greedy, bit manipulation and DP foundations	Classic optimization problems
10	Advanced DP and graph algorithms	Shortest path, topological sort, LCS, knapsack
11	Timed coding tests and optimization	Mixed problem sets with complexity explanation
12	Projects, mock interviews and final revision	Final project demo and interview readiness score

Project-Based Learning

Projects prove that the candidate can apply programming concepts beyond isolated problems. Each project should be version-controlled, explained and tested.

Language	Beginner Projects	Intermediate Projects	Advanced Projects
C	Calculator, marksheet, number guessing	Banking with structures, file employee records	Linked-list contact book, inventory with files
C++	Student class, shopping cart	Library system, STL inventory, parking lot	LRU cache, graph route finder, CP toolkit
Python	Calculator, to-do app, word counter	CSV analytics, expense tracker, JSON processor	Resume analyzer, automation scripts, API mini app

Project validation checklist: working demo, readable code, file or data handling where relevant, error handling, test cases and a clear resume explanation.

Candidate Assessment Rubric

Use this rubric to score candidate readiness before interviews or placement drives.

Assessment Area	Weight	What to check
Syntax and fundamentals	15%	Can write correct basic programs without copying.
Logic building	20%	Can dry-run, handle conditions and loops, and solve small problems.
Data structures	20%	Can choose and implement arrays, stack, queue, hash map, tree and graph.
Algorithms	20%	Can explain and code search, sort, recursion, DP, greedy and graph patterns.
Debugging	10%	Can find boundary cases, invalid input, off-by-one and memory issues.
Code readability	5%	Uses names, functions, comments and clean formatting.
Project implementation	10%	Can build and explain working mini projects.

Final readiness levels:

- Below 50: needs foundation rebuild and daily coding discipline.
- 50 to 70: has basics but needs structured DSA and timed practice.
- 70 to 85: interview-ready for many service-based roles with mock practice.

- 85 plus: strong candidate for tougher coding rounds and advanced interviews.

Final Revision Checklist

Checklist Item	Ready when candidate can...
Syntax	Write basic programs in C/C++/Python without copying.
Dry run	Trace variables, loops and recursion on paper.
Debugging	Find off-by-one, invalid input and memory-related errors.
DSA	Choose array, hash map, stack, queue, tree or graph based on the problem.
Algorithms	Explain brute force, optimized approach, complexity and edge cases.
Projects	Demo at least one project per language track or one strong combined project.
Interview communication	Explain approach before coding and test after coding.

Positioning: C/C++/Python Programming Mastery for Coding Interviews - from programming fundamentals to DSA, projects and mock interviews.